

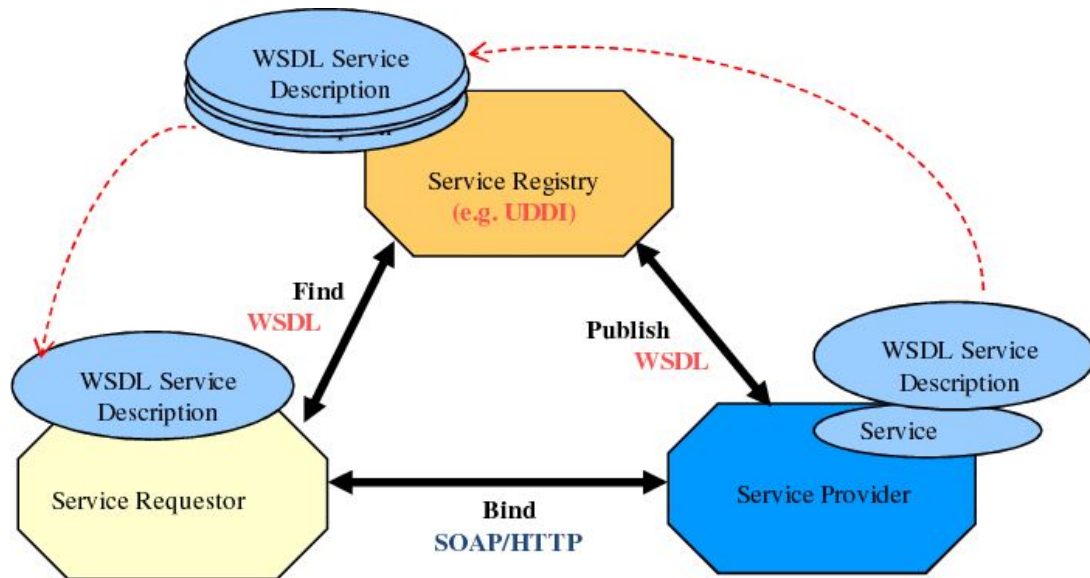
Python p/ Processamento de Dados

Etapa 8

Professor(a):

E-mail: marcelo.hama@prof.infnet.edu.br

Consumo de Dados e APIs Web



Arquitetura SOAP e exemplo de WSDL.

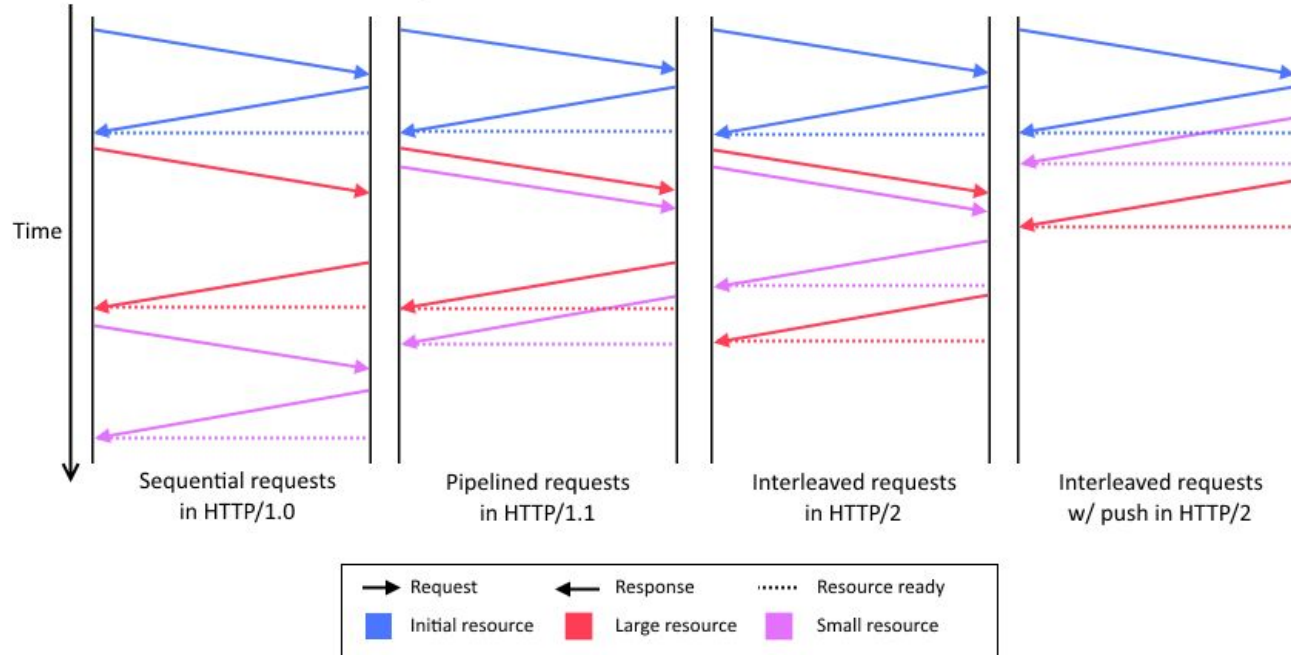
Fonte: <https://www.researchgate.net/>

```

<?xml version="1.0" encoding="utf-8"?>
<wsdl:definitions xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" ...>
  <wsdl:types>
    <s:schema elementFormDefault="qualified" targetNamespace="http://tempuri.org/">
      ...
      <s:element name="GetWordListResponse">
        <s:complexType>
          <s:sequence>
            <s:element minOccurs="0" maxOccurs="1" name="GetWordListResult" type="tns:ArrayOfString" />
          </s:sequence>
        </s:complexType>
      </s:element>
    </s:schema>
  </wsdl:types>
  <wsdl:message name="GetWordListSoapIn">
    <wsdl:part name="parameters" element="tns:GetWordList" />
  </wsdl:message>
  <wsdl:message name="GetWordListSoapOut">
    <wsdl:part name="parameters" element="tns:GetWordListResponse" />
  </wsdl:message>
  <wsdl:portType name="AutoWebServiceSoap">
    <wsdl:operation name="GetWordList">
      <wsdl:input message="tns:GetWordListSoapIn" />
      <wsdl:output message="tns:GetWordListSoapOut" />
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:binding name="AutoWebServiceSoap" type="tns:AutoWebServiceSoap">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" />
    <wsdl:operation name="GetWordList">
      <soap:operation soapAction="http://tempuri.org/GetWordList" style="document" />
      <wsdl:input>
        <soap:body use="literal" />
      </wsdl:input>
      <wsdl:output>
        <soap:body use="literal" />
      </wsdl:output>
    </wsdl:operation>
  </wsdl:binding>
  <wsdl:service name="AutoWebService">
    <wsdl:port name="AutoWebServiceSoap" binding="tns:AutoWebServiceSoap">
      <soap:address location="http://www.vietnamofficeexpress.com/AutoWebService.asmx" />
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
  
```

Consumo de Dados e APIs Web

Comparison of time to fetch HTTP resources

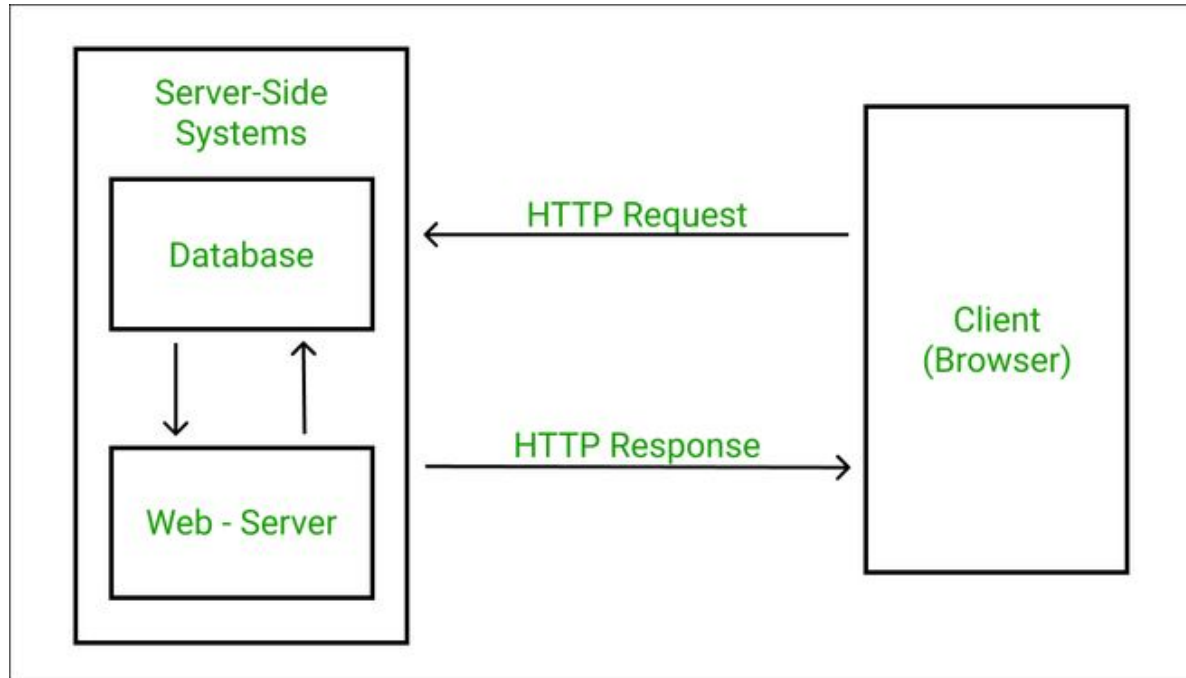


Milestones entre HTTP/1.0 e HTTP /2. Fonte:

<https://medium.com/@sandeep4.verma/http-1-to-http-2-to-http-3-647e73df67a8>

Consumo de Dados e APIs Web

Diagrama de Uma Arquitetura Cliente/Servidor Padrão



Consumo de Dados e APIs Web

PARTE 1

Métodos de Conexão e Transmissão na Web

Métodos de Conexão e Transmissão na Web

REST (Representational State Transfer)

É um modelo de arquitetura e não uma linguagem ou tecnologia de programação. Os conceitos do REST foram submetidos à tese de doutorado de Roy Fielding nos anos 2000, onde o princípio fundamental é usar o protocolo HTTP para comunicação de dados.

O REST precisa que um cliente faça uma requisição para o servidor para enviar ou modificar dados. Uma requisição consiste em:

- Um método HTTP, que define a operação o servidor fará: GET, POST, PUT, DELETE, ou PATCH;
- Um header, com o cabeçalho da requisição que passa informações sobre a requisição;
- Um path (caminho ou rota) para o servidor;
- Informação no corpo da requisição, sendo esta informação opcional.

Respostas HTTP

- 2XX: Requisição processada
- 3XX: Requisição com necessidade de correções ou modificações
- 4XX: Erros na requisição do cliente
- 5XX: Erros de servidor

Métodos de Conexão e Transmissão na Web

HTTP Method	CRUD	Collection Resource (e.g. /users)	Single Resource (e.g. /users/123)
POST	Create	201 (Created), 'Location' header with link to /users/{id} containing new ID	Avoid using POST on a single resource
GET	Read	200 (OK), list of users. Use pagination, sorting, and filtering to navigate big lists	200 (OK), single user. 404 (Not Found), if ID not found or invalid
PUT	Update/Replace	405 (Method not allowed), unless you want to update every resource in the entire collection of resource	200 (OK) or 204 (No Content). Use 404 (Not Found), if ID is not found or invalid
PATCH	Partial Update/Modify	405 (Method not allowed), unless you want to modify the collection itself	200 (OK) or 204 (No Content). Use 404 (Not Found), if ID is not found or invalid
DELETE	Delete	405 (Method not allowed), unless you want to delete the whole collection — use with caution	200 (OK). 404 (Not Found), if ID not found or invalid

Métodos de Conexão e Transmissão na Web

Obtendo Dados por uma API

Disponibilizar dados por meio de uma API é bastante comum, seguindo uma tendência de desacoplar aplicações em serviços que se comuniquem via APIs.

As APIs podem funcionar de diversas maneiras, mas geralmente operam sobre protocolos HTTP/HTTPS regulares, utilizando os verbos HTTP padrão: GET, POST, PUT e DELETE.

Obter dados dessa forma é muito semelhante a recuperar um arquivo, mas os dados não estão em um arquivo estático. Em vez de a aplicação servir arquivos estáticos que contêm os dados, ela consulta alguma outra fonte de dados e, em seguida, reúne e fornece os dados dinamicamente sob demanda.

Métodos de Conexão e Transmissão na Web

Obtendo Dados por uma API

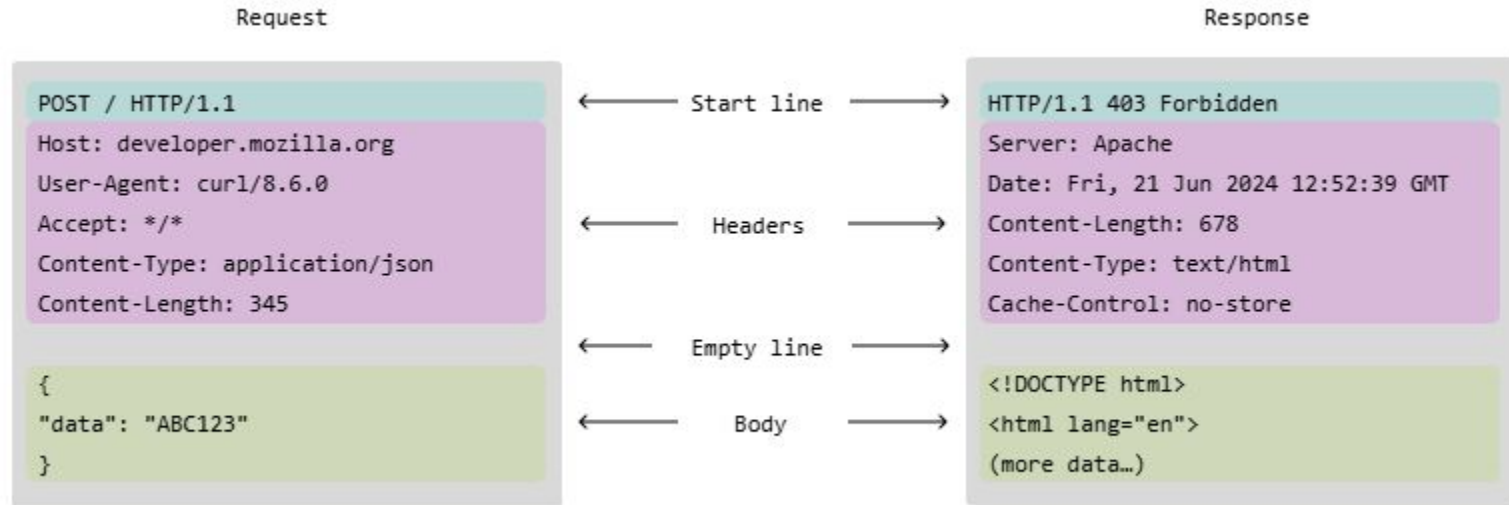
Embora haja muita variação nas maneiras de configurar uma API, uma das mais comuns é uma interface RESTful (Representational State Transfer), que opera sobre os mesmos protocolos HTTP/HTTPS da web.

Existem inúmeras variações de como uma API pode funcionar, mas, geralmente, os dados são obtidos por meio de uma requisição GET, que é o método usado pelo seu navegador para solicitar uma página da web.

Ao fazer uma requisição GET, os parâmetros para selecionar os dados desejados são frequentemente adicionados à URL em uma string de consulta.

Métodos de Conexão e Transmissão na Web

Anatomia de Uma Mensagem HTTP



Consumo de Dados e APIs Web

PARTE 2

Requisições Web GET Usando Requests

Requisições Web GET Usando Requests

Obtendo Dados por uma API

Podemos testar obtendo a previsão do tempo atual em Chicago do Open-Meteo.com, usando a API <https://mng.bz/BX9g> como URL. Os itens após o "?" são parâmetros da string de consulta que especificam a latitude e a longitude da localização e os dois campos que estamos solicitando: temperatura e velocidade do vento.

Para esta API, as informações serão retornadas em JSON. Observe que os elementos da string de consulta são separados por "&". Quando você souber a URL a ser usada, poderá utilizar a biblioteca requests para buscar dados de uma API e processá-los instantaneamente ou salvá-los em um arquivo para processamento posterior.

Requisições Web GET Usando Requests

Recuperando Dados com GET

```
import json
import requests

BASE_URL = (
    "https://api.open-meteo.com/v1/forecast?" +
    "latitude=41.882&longitude=-87.623&" +
    "current=temperature_2m,wind_speed_10m")

response = requests.get(BASE_URL)
print(f"Status Code: {response.status_code}")

info = json.dumps(response.json(), indent=4)
print(f"Response JSON: {info}")
```

Idealmente, você deve escapar espaços e a maioria dos sinais de pontuação nos parâmetros de consulta, pois esses elementos não são permitidos em URLs.

Isso não é absolutamente necessário, pois a biblioteca requests e muitos navegadores fazem o escape automático de URLs.

Requisições Web GET Usando Requests

A Biblioteca Requests

A biblioteca requests é a solução padrão do Python para fazer requisições HTTP de forma simples e legível. É a biblioteca mais usada para consumir APIs REST em Python e serve de base para ferramentas como httpx e aiohttp (versão assíncrona).

Principais Formas dos Métodos HTTP

```
import requests
```

```
requests.get("https://api.exemplo.com/dados")      # buscar dados
requests.post("https://api.exemplo.com/criar")      # enviar dados
requests.put("https://api.exemplo.com/atualizar/1") # substituir recurso
requests.patch("https://api.exemplo.com/editar/1") # atualizar parcialmente
requests.delete("https://api.exemplo.com/remover/1") # excluir recurso
```

Requisições Web GET Usando Requests

Parâmetros de URL, Headers, e Autenticação

```
# parâmetros na URL (?pagina=2&limite=10)
r = requests.get(url, params={"pagina": 2, "limite": 10})
# headers customizados
r = requests.get(url, headers={"Authorization": "Bearer token123"})
# autenticação básica
r = requests.get(url, auth=("usuario", "senha"))
```

Parâmetros de URL, Headers, e Autenticação

```
# JSON no corpo da requisição
r = requests.post(url, json={"nome": "Ana", "idade": 30})
# formulário (form-data)
r = requests.post(url, data={"campo": "valor"})
# upload de arquivo
r = requests.post(url, files={"arquivo": open("foto.jpg", "rb")})
```

Requisições Web GET Usando Requests

Trabalhando com as Respostas

```
r = requests.get("https://api.exemplo.com/dados")
r.status_code      # 200, 404, 500 ...
r.text             # corpo como string
r.json()           # corpo como dict Python (se for JSON)
r.headers          # headers da resposta
r.content          # corpo como bytes (para imagens, PDFs etc.)
r.url              # URL final (após redirects)
r.elapsed          # tempo de resposta
```

Reaproveitamento de Sessões

```
# Session mantém cookies, headers e conexão TCP entre requisições
with requests.Session() as s:
    s.headers.update({"Authorization": "Bearer token"})
    s.get("https://api.exemplo.com/perfil")
    s.post("https://api.exemplo.com/logout")
```


Requisições Web GET Usando Requests

Tratamento de Erros

```
import requests
url = "https://example.com"
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status()
    data = response.json()
except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err}") # e.g., 404 Client Error
except requests.exceptions.ConnectionError as conn_err:
    print(f"Connection error occurred: {conn_err}") # e.g., DNS failure
except requests.exceptions.Timeout as timeout_err:
    print(f"Timeout error occurred: {timeout_err}") # e.g., Server took too long
except requests.exceptions.JSONDecodeError as json_err:
    print(f"Failed to parse JSON response: {json_err}") # Invalid body format
except requests.exceptions.RequestException as req_err:
    print(f"An ambiguous error occurred: {req_err}") # Catch-all fallback
```

Faixa	Significado	Exemplos
2xx	Sucesso	200 OK, 201 Created, 204 No Content
3xx	Redirecionamento	301 Moved, 302 Found
4xx	Erro do cliente	400 Bad Request, 401, 403, 404
5xx	Erro do servidor	500 Internal Error, 503 Unavailable

Requisições Web GET Usando Requests

Resumo das Funcionalidades do requests

Recurso

params

json

headers

auth

timeout

files

raise_for_status()

Session

O que faz

Query string na URL

Corpo JSON na requisição

Headers customizados

Autenticação Basic ou Bearer

Limite de espera

Upload de arquivos

Exceção automática em erros HTTP

Conexão persistente com cookies

Consumo de Dados e APIs Web

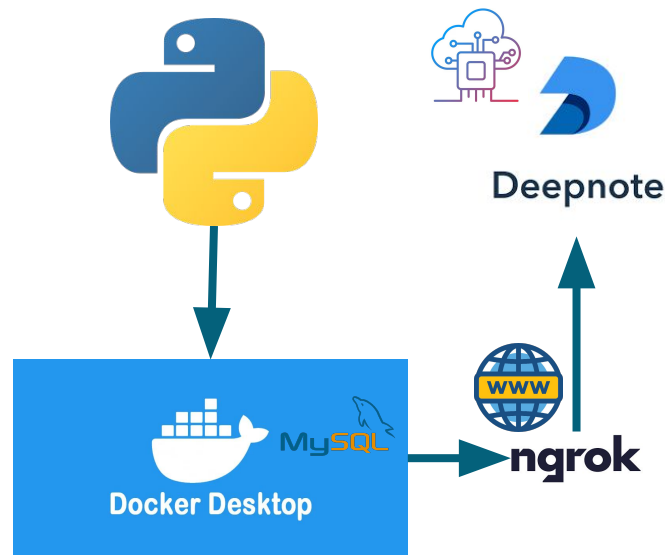
PARTE 3

Interpretando o Web Response

Interpretando o Web Response

Stack Laboratorial

- Conta Deepnote para conexão em nuvem e integração com camada de analytics;
- Tunneling com ngrok, para publicação de um servidor público Python;
- App Python, com receita dockerfile rodando e configs para expor porta 5000;
- Plataforma Docker instalada na máquina local, disponibilizando um contêiner python:3.11-slim;



Interpretando o Web Response

Configuração do Servidor Python em Container Docker Desktop

1. Instalar o Docker Desktop v4.32.0 (157355):
<https://www.docker.com/products/docker-desktop/>
2. Criar conta na HUB Docker:
<https://login.docker.com/>
3. No prompt de Comando:
 - a. Criar uma ponte de rede para expor o contêiner:
\$ docker network create --driver bridge infnet-network
 - b. Construir o contêiner através da receita Dockerfile
\$ docker build -t python-rest-api .
 - c. Executar o contêiner com a aplicação, na porta 5000
\$ docker run -d -p 5000:5000 --name my-api-server python-rest-api

Interpretando o Web Response

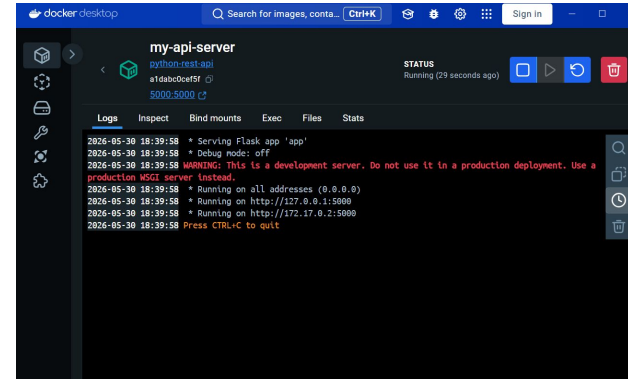
Configuração do Servidor Python em Container Docker Desktop

```

[+] Building 1.8s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 456B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.7s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                    0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb8409 0.0s
=> CACHED [2/5] WORKDIR /app                                       0.0s
=> CACHED [3/5] COPY requirements.txt .                             0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                       0.0s
=> exporting image                                                0.0s
=> => exporting layers                                           0.0s
=> writing image sha256:d80clccf9ac24e8f14245e1642d7c51b1cd0ea0e0757720ce44cdfd6fb73cdd 0.0s
=> => naming to docker.io/library/python-rest-api                 0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/qg1517sh66unwjcmo
xuhgb19z

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview
PS C:\Users\marce\Downloads\python\Python_para_Processamento_de_Dados_08_exe>
  
```



Search		Only running				
☐	Port(s)	Name	Container ID	Image	C	Actions
☒	5000:5000 ↗	my-api-server	28291f87a2f2	python-rest	☐	⋮ 🗑️

Interpretando o Web Response

Configuração do ngrok e tunneling para a rede pública

```
C:\Program Files\WindowsAp  x  +  v  -  □  x
ngrok  (Ctrl+C to quit)

Request early access to new features: https://dashboard.ngrok.com/early-access

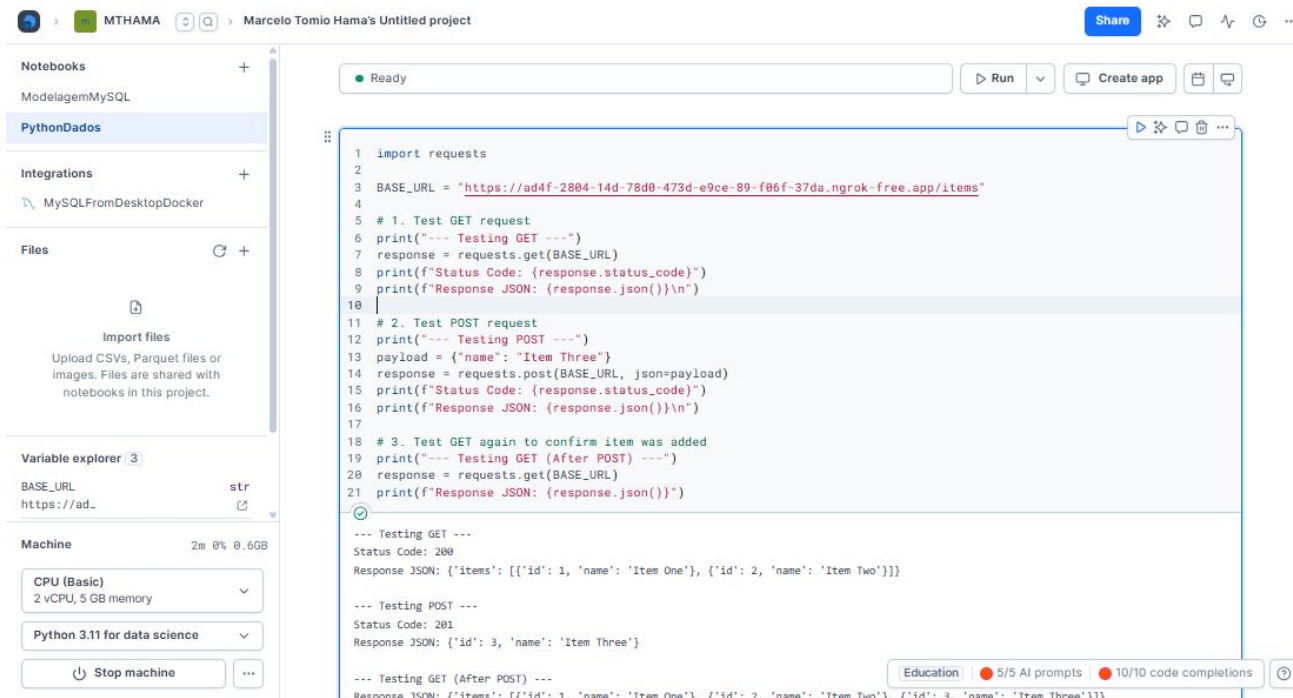
Session Status      online
Account             hama.marcelo.ti@gmail.com (Plan: Free)
Version             3.39.1-msix-stable
Region              South America (sa)
Latency              25ms
Web Interface       http://127.0.0.1:4040
Forwarding           https://ad4f-2804-14d-78d0-473d-e9ce-89-f06f-37da.ngrok-free.app -> http://localhost:5000

Connections          ttl    opn    rt1    rt5    p50    p90
                    0      0      0.00   0.00   0.00   0.00
```

1. Webpage do ngrok e instruções (necessário configurar chave de API/token):
<https://ngrok.com/download/>
2. No prompt de Comando:
 - a. Exposição do contêiner Python server através da porta http 5000:
\$ ngrok http 5000
 - b. Anote a URL de forwarding e use-a no Deepnote.

Interpretando o Web Response

Integração com o Deepnote



The screenshot displays the Deepnote web interface for a project named "Marcelo Tomio Hama's Untitled project". The left sidebar shows a file explorer with a notebook named "PythonDados" and a variable explorer with a variable "BASE_URL" set to "https://ad...". The main area shows a Python script that tests a REST API using the 'requests' library. The script performs a GET request, a POST request with a payload, and another GET request to verify the data. The output shows the status codes and JSON responses for each request.

```
1 import requests
2
3 BASE_URL = "https://ad4f-2884-14d-78d0-473d-e9ce-89-f06f-37da.ngrok-free.app/items"
4
5 # 1. Test GET request
6 print("--- Testing GET ---")
7 response = requests.get(BASE_URL)
8 print(f"Status Code: {response.status_code}")
9 print(f"Response JSON: {response.json()}\n")
10
11 # 2. Test POST request
12 print("--- Testing POST ---")
13 payload = {"name": "Item Three"}
14 response = requests.post(BASE_URL, json=payload)
15 print(f"Status Code: {response.status_code}")
16 print(f"Response JSON: {response.json()}\n")
17
18 # 3. Test GET again to confirm item was added
19 print("--- Testing GET (After POST) ---")
20 response = requests.get(BASE_URL)
21 print(f"Response JSON: {response.json()}")
```

--- Testing GET ---
Status Code: 200
Response JSON: {'items': [{'id': 1, 'name': 'Item One'}, {'id': 2, 'name': 'Item Two'}]}

--- Testing POST ---
Status Code: 201
Response JSON: {'id': 3, 'name': 'Item Three'}

--- Testing GET (After POST) ---
Response JSON: {'items': [{'id': 1, 'name': 'Item One'}, {'id': 2, 'name': 'Item Two'}, {'id': 3, 'name': 'Item Three'}]}

Interpretando o Web Response

Aplicação Servidor

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Persistencia de dados em memoria
data_store = [
    {"id": 1, "name": "Item One"},
    {"id": 2, "name": "Item Two"}
]

@app.route('/items', methods=['GET'])
def get_items():
    return jsonify({"items": data_store}), 200
```

```
@app.route('/items', methods=['POST'])
def add_item():
    new_item = request.get_json()
    if not new_item or 'name' not in new_item:
        return jsonify({"error": "Bad Request",
            "message": "Missing 'name' field"}), 400

    new_item['id'] = len(data_store) + 1
    data_store.append(new_item)
    return jsonify(new_item), 201

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Interpretando o Web Response

Integração com o Deepnote

```
import requests
```

```
BASE_URL = "<URL_NGROK>/items"
```

```
# 1. Test GET request
```

```
print("--- Testing GET ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 2. Test POST request
```

```
print("--- Testing POST ---")
```

```
payload = {"name": "Item Three"}
```

```
response = requests.post(BASE_URL, json=payload)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 3. Test GET again to confirm item was added
```

```
print("--- Testing GET (After POST) ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Response JSON: {response.json()}\n")
```

Interpretando o Web Response

Interpretação do *Response*

Quando o *requests* faz uma requisição, retorna um objeto *Response* com tudo que o servidor devolveu. Saber interpretar esse objeto é essencial para trabalhar com APIs.

Sempre verifique *status_code* ou use *raise_for_status()*, e só acesse *.json()* quando tiver certeza que o *Content-Type* é *application/json*.

O Status Code é o primeiro diagnóstico.

```
import requests

BASE_URL = "<URL_NGROK>/items"
r = requests.get(BASE_URL)

print(r.status_code)    # 200
print(r.ok)             # True se status < 400

# Verificação manual
if r.status_code == 200:
    print("Sucesso")
elif r.status_code == 404:
    print("Não encontrado")
elif r.status_code == 401:
    print("Não autorizado")
elif r.status_code == 500:
    print("Erro no servidor")
```

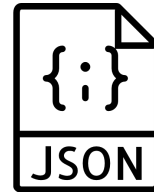
Interpretando o Web Response

Acessando o Corpo da Resposta

```
# FORMA 1: API que retorna JSON: sempre use .json()
r = requests.get(BASE_URL)
usuario = r.json()
print(usuario["name"])          # The Octocat
print(usuario["public_repos"])
```

```
# FORMA 2: Página HTML: use .text
r = requests.get(BASE_URL)
html = r.text
```

```
# FORMA 3: Arquivo binário: use .content
r = requests.get(BASE_URL)
with open("foto.jpg", "wb") as f:
    f.write(r.content)
```



Interpretando o Web Response

Headers da Resposta

```
# acessar todos os headers
print(dict(r.headers))

# acessar header específico (case-insensitive)
print(r.headers["Content-Type"])

# application/json

print(r.headers.get("X-Rate-Limit", "sem limite"))

# header opcional com valor padrão
```

```
# tipo do conteúdo
r.headers["Content-Type"]

# tamanho em bytes
r.headers["Content-Length"]

# política de cache
r.headers["Cache-Control"]

# requisições restantes na API
r.headers["X-RateLimit-Remaining"]

# cookies definidos pelo servidor
r.headers["Set-Cookie"]
```

Interpretando o Web Response

Encodings (Leitura Correta)

```
# encoding detectado automaticamente pelo
requests
print(r.encoding) # 'utf-8' ou 'ISO-8859-1'

# forçar encoding correto se detectado errado
r.encoding = "utf-8"
print(r.text) # agora lê com o encoding correto

# encoding real detectado pela biblioteca chardet
print(r.apparent_encoding) # baseado no conteúdo
```

Outros Parâmetros da Resposta

```
# cookies recebidos na resposta
print(r.cookies)

# URL final após todos os redirects
print(r.url)

# 200 (do destino final)
print(r.status_code)

# histórico de redirects
for resp in r.history:
    print(resp.status_code, resp.url)

# tempo de resposta, em segundos
print(r.elapsed.total_seconds())
```

Interpretando o Web Response

Tratamento de Resposta (Robusto)

```
import requests
import json
```

```
BASE_URL = "<NGROK_URL>/items"
```

```
resultado = chamar_api(BASE_URL)
print(json.dumps(resultado["items"], indent=4))
```

```
def chamar_api(url):
    try:
        r = requests.get(url, timeout=10)
        r.raise_for_status()
    except requests.exceptions.Timeout:
        return {"erro": "timeout"}
    except requests.exceptions.HTTPError as e:
        return {"erro": f"{r.status_code}: {e}"}
    except requests.exceptions.JSONDecodeError:
        return {"erro": "JSON inválido"}
    except requests.exceptions.ConnectionError:
        return {"erro": "falha de conexão"}
    # verificar Content-Type antes de .json()
    cont_type = r.headers.get("Content-Type", "")
    if "application/json" in cont_type:
        return r.json()
    else:
        return {"texto": r.text}
```

Consumo de Dados e APIs Web

PARTE 4

Requisições Web POST com Requests

Requisições Web POST com Requests

O método *POST* do módulo *requests* recebe dados adicionais como um dicionário Python. O *POST* codifica e serializa os dados, calculando e inserindo cabeçalhos adicionais, como *content-length* e o método de codificação, montando a requisição *HTTP* de acordo com os padrões estabelecidos, e estabelecendo a conexão de rede com o envio da requisição. Novamente, a resposta de uma requisição *POST* é retornada como um objeto *Response*, que pode ser consultada para obter o status e os resultados.

```
import requests
base = 'https://pubchem.ncbi.nlm.nih.gov'
endpoint = base + '/rest/pug/compound/cid/property/MolecularFormula,MolecularWeight/CSV'
data = {'cid': '679,3776,11551'}
resp = requests.post(endpoint, data=data)
print(resp.headers['content-type'])
print(resp.text)
```

Requisições Web POST com Requests

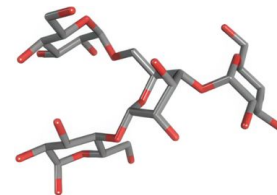
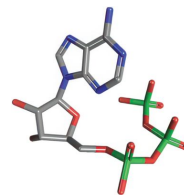
POST com Arquivos Binários

```
import requests
# Request and save binary image of molecule
def fetch_and_save_image(mname, fname):
    domain = 'https://pubchem.ncbi.nlm.nih.gov'
    url = (domain +
          f'/rest/pug/compound/name/{mname}' +
          '/record/PNG?record_type=3d')
    # Perform the HTTP request and check status
    resp = requests.get(url)
    if resp.status_code != 200:
        print(f'Unexpected response')
    # Save returned binary data to a file
    with open(fname, 'wb') as file:
        for chunk in resp.iter_content(chunk_size=128):
            file.write(chunk)
    print(f'Image saved to {fname}')
```

```
if __name__ == '__main__':
    # Get molecule name parameter
    mname = 'atp'
    #mname = 'glycogen'

    fname = f'{mname}.png'

    # Invoke function
    fetch_and_save_image(mname, fname)
```



Requisições Web POST com Requests

Customizando o Objeto da Requisição

As funções do módulo *requests*, como *GET* e *POST*, começam criando um objeto *Request* interno a partir do parâmetro *URL* fornecido e do dicionário de dados opcional. Na maioria das vezes, isso é tratado perfeitamente pelo módulo *requests*.

Mas, ocasionalmente, pode ser necessário personalizar o objeto *Request* antes que a requisição *HTTP* seja realizada.

Quase tudo que pode ser personalizado está disponível como parâmetros de palavra-chave que podem ser passados para as funções de nível superior do módulo *requests*.

Keyword parameter	Description
<code>data</code>	Python dictionary formatted and included in request body.
<code>params</code>	Python dictionary transformed into query string and added to URL.
<code>headers</code>	Python dictionary of headers to add to Request object.
<code>auth</code>	(username, password) tuple used for required web service authorization.
<code>proxies</code>	Python dictionary that maps protocol name to the URL of a proxy server.

Requisições Web POST com Requests

Envio com JSON (comum em APIs REST)

```
import requests
address = "<NGROK_URL>/items"
r = requests.post(
    url      = address,
    json     = {"name": "Item Three"},
    headers  = {"Authorization": "Bearer
token123"},
    timeout  = 10
)
# verificar o que foi enviado
print(r.request.body)
print(r.request.headers["Content-Type"])
print(r.status_code)
print(r.json())
```

Envio Form Data (HTML tradicionais)

```
import requests
address = "<NGROK_URL>/items"
content = {"name": "Item Three"}
r = requests.post(
    address,
    data=content
)
print(r.request.body)
print(r.request.headers["Content-Type"])
print(r.status_code)
print(r.text)
```

Requisições Web POST com Requests

Lidando com a Resposta do POST

```
# status esperado para POST bem-sucedido
if r.status_code == 201:          # Created
    novo_usuario = r.json()
    print(f"ID criado: {novo_usuario['id']}")
elif r.status_code == 200:       # OK (alguns servidores usam 200)
    print(r.json())
elif r.status_code == 400:       # Bad Request
    print("Dados inválidos:", r.json())
elif r.status_code == 409:       # Conflict
    print("Recurso já existe:", r.json())
elif r.status_code == 422:       # Unprocessable Entity
    print("Validação falhou:", r.json())
```

Requisições Web POST com Requests

Aplicação Servidor

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Persistencia de dados em memoria
data_store = [
    {"id": 1, "name": "Item One"},
    {"id": 2, "name": "Item Two"}
]

@app.route('/items', methods=['GET'])
def get_items():
    return jsonify({"items": data_store}), 200
```

```
@app.route('/items', methods=['POST'])
def add_item():
    new_item = request.get_json()
    if not new_item or 'name' not in new_item:
        return jsonify({"error": "Bad Request",
            "message": "Missing 'name' field"}), 400

    new_item['id'] = len(data_store) + 1
    data_store.append(new_item)
    return jsonify(new_item), 201
```

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Requisições Web POST com Requests

Integração com o Deepnote

```
import requests
```

```
BASE_URL = "<URL_NGROK>/items"
```

```
# 1. Test GET request
```

```
print("--- Testing GET ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 2. Test POST request
```

```
print("--- Testing POST ---")
```

```
payload = {"name": "Item Three"}
```

```
response = requests.post(BASE_URL, json=payload)
```

```
print(f"Status Code: {response.status_code}")
```

```
print(f"Response JSON: {response.json()}\n")
```

```
# 3. Test GET again to confirm item was added
```

```
print("--- Testing GET (After POST) ---")
```

```
response = requests.get(BASE_URL)
```

```
print(f"Response JSON: {response.json()}\n")
```

Consumo de Dados e APIs Web

PARTE 5

Manipulando Respostas em JSON

Manipulando Respostas em JSON

O Método `.json()` como Ponto de Entrada

Chamar `.json()` em uma resposta não-JSON gera *JSONDecodeError*. A forma correta é verificar primeiro:

```
import requests
address = "<NGROK_URL>/items"
r = requests.get(address)
# verificação pelo Content-Type
content_type = r.headers.get("Content-Type", "")
if "application/json" in content_type:
    dados = r.json()
else:
    print("Resposta não é JSON:", r.text[:200])
# verificação pelo status + content-type juntos
if r.ok and "application/json" in r.headers.get("Content-Type", ""):
    dados = r.json()
    print(dados)
```

Manipulando Respostas em JSON

Navegando na Resposta Simples

É possível navegar pelo objeto retornado de diferentes maneiras:

```
import requests
address = "<NGROK_URL>/items"

r = requests.get(address)
u = r.json()

# acesso direto por chave
print(u["items"])
# acesso em objetos aninhados
print(u["items"][0])
# .get() – retorna None se a chave não existir
print(u["items"][0].get("name"))
print(u["items"][0].get("address"))
```

Manipulando Respostas em JSON

Navegando na Resposta como Lista

Outra maneira de navegação e manipulação é por meio da listagem:

```
import requests
address = "<NGROK_URL>/items"

r = requests.get(address)
u = r.json()

print(type(u))
print(type(u["items"]))
print(len(u["items"]))
```

```
# acessar por índice
print(u["items"][0]["name"]) # Leanne Graham

# iterar sobre todos
for v in u["items"]:
    print(f"{v['id']:2d} {v['name']:<25}")
```

Manipulando Respostas em JSON

Filtragem e Transformação

```
import requests
address = "<NGROK_URL>/items"
r = requests.get(address)
registros = r.json()["items"]

# filtrar por critério
dados = [u for u in registros if u["name"] == "Item Three"]
# extrair apenas campos relevantes
dados = [{"id": u["id"], "nome": u["name"]}
         for u in registros]
# ordenar pelo nome
dados = sorted(registros, key=lambda u: u["name"])
# buscar um único registro
dados = next((u for u in registros if u["name"] == "Item One"), None)

print(dados)
```

Manipulando Respostas em JSON

Salvando e Carregando em Arquivos

```
import json
import requests
address = "<NGROK_URL>/items"

# salvar resposta em arquivo
r = requests.get(address)
with open("usuarios.json", "w", encoding="utf-8") as f:
    json.dump(r.json()["items"], f, indent=2, ensure_ascii=False)

# carregar do arquivo depois
with open("usuarios.json", "r", encoding="utf-8") as f:
    usuarios = json.load(f)

print(f"{len(usuarios)} usuários carregados do arquivo")
```

Manipulando Respostas em JSON

Conversão para Outras Estruturas

```
import json
import requests
import pandas as pd
```

```
address = "<NGROK_URL>/items"
```

```
r = requests.get(address)
usuarios = r.json()["items"]
```

```
# — para dicionário indexado por ID —
por_id = {u["id"]: u for u in usuarios}
print(por_id[3]["name"])
```

```
# — para conjunto de e-mails —
names = {u["name"].lower() for u in usuarios}
```

```
# — para DataFrame pandas —
df = pd.DataFrame(usuarios)
print(df[["id", "name"]].head())
```

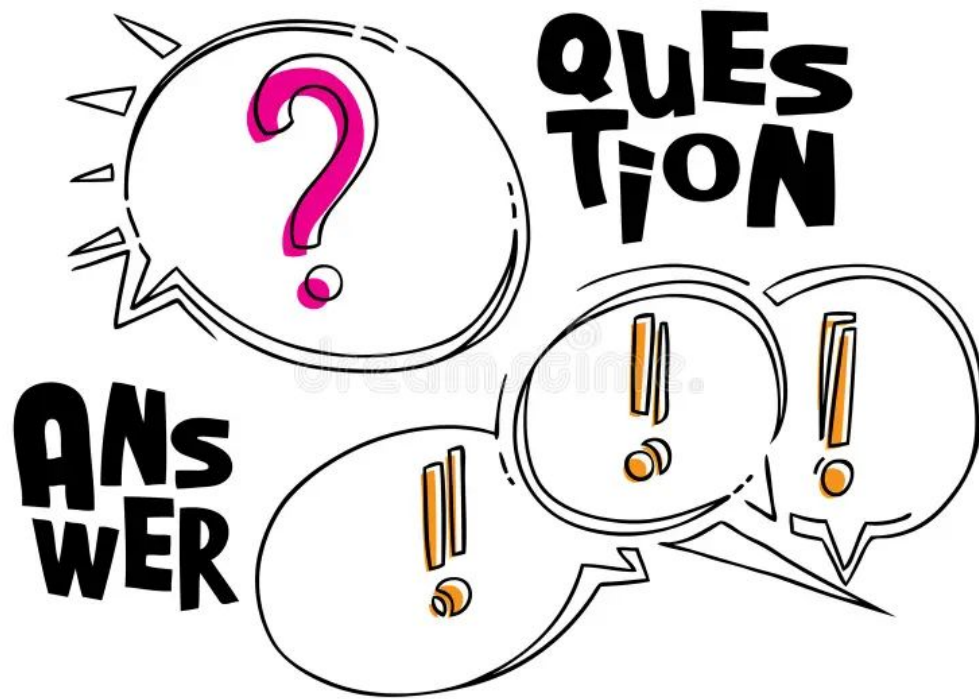
```
# normalizar colunas aninhadas
df_flat = pd.json_normalize(usuarios)
print(df_flat["name"].tolist())
```

Manipulando Respostas em JSON

Resumo das Operações

Operação	Como fazer
Converter resposta em dict/list	<code>r.json()</code>
Acessar chave com segurança	<code>.get("chave", valor_padrão)</code>
Navegar em objetos aninhados	<code>.get("a", {}).get("b", {}).get("c")</code>
Filtrar lista	<code>[x for x in lista if condição]</code>
Buscar um item	<code>next((x for x in lista if cond), None)</code>
Verificar antes de parsear	<code>"application/json" in r.headers["Content-Type"]</code>
Salvar em arquivo	<code>json.dump(r.json(), f, indent=2)</code>
Converter para DataFrame	<code>pd.json_normalize(r.json())</code>
Tratar resposta malformada	<code>try: r.json() except json.JSONDecodeError</code>

Dúvidas e Perguntas



Referências Bibliográficas

- Get Programming, Ana Bell, Manning Publications;
- Introducing Python, Bill Lubanovic, O'Reilly Media, Inc;
- Learn to Code by Solving Problems, Daniel Zingaro, No Starch Press queue;
- Python Crash Course, Eric Matthes, No Starch Press;
- Streamlining Your Research Laboratory with Python, Mark F. Russo, William Neil, Wiley;
- The Quick Python Book, Naomi Ceder, Manning Publications;
- <https://deepnote.com/docs/incoming-connections>;
- <https://ai.google.dev/gemini-api/docs>;
- <https://developers.openai.com/api/docs/libraries>.